CHESS AI USING ADVERSARIAL SEARCH

Kaitlyn Cornish, Eli Lawerence, Alice Hui

WSU

CPTS 440

Contents

I. Introduction 2

II. Problem Statement 2

III. Approach Overview 3

IV. Algorithms 3

IV.1. Minimax 3

IV.2 Alpha-Beta Pruning 4

V. Evaluation Function 5

VI. Experimental Setup 5

VII. Results and Analysis 6

IX. Conclusion 7

X. Future Work 8

I. Introduction

The ability to make good decisions in competitive environments is a core problem in artificial intelligence, especially in games where two players are constantly truing to out-strategize each other. Games like chess are a classic example of this because they have clear rules, no randomness, and require planning several moves ahead. Even though the rules are simple, the number of possible board positions grows extremely quickly, which makes it difficult to evaluate every possible outcome.

One common way to approach this problem is through adversarial search, where an AI assumes that its opponent is also making the best possible decisions. This creates a back- and-forth structure where each move must account for how the opponent might respond, However, because of the large number of possible moves in chess, these algorithms can become very expensive in terms of computation time.

In this project, we designed and implemented a chess-playing AI using two classical adversarial search algorithms, minimax and alpha-beta pruning. The goal was not only to create a functional AI that can play chess, but also to analyze how these algorithms perform in practice. In particular, we focused on how search depth impacts performance, how many game states are explored, and how alpha-beta pruning improves efficiency compared to standard minimax.

Overall, this project explores the trade-off between decision quality and computational cost, and demonstrates how optimization techniques can significantly improve the practicality of adversarial search in complex games like chess.

II. Problem Statement

Designing an AI that can play chess effectively is challenging because of the sheer number of possible moves and game states. At any point in the game, a player may have dozens of legal moves, and each of those leads to even more possibilities in future turns. This creates a rapidly growing search space, making it impractical to evaluate every possible outcome all the way to the end of the game.

The main problem in this project is determining how to make strong decisions within these constraints. A deeper search generally leads to better decisions, since the AI can look further ahead, but it also requires significantly more computation time. This creates a trade-off between accuracy and efficiency that must be carefully managed.

To address this, we focus on implementing and comparing two adversarial search algorithms: minimax and alpha-beta pruning. The goal is to evaluate how efficiently each algorithm can explore the search space and how their performance changes as the search depth increases. Ultimately, we aim to determine how to produce strong moves while keeping computation time at a reasonable level.

III. Approach Overview

To address the challenges of decision-making in chess, we designed a system that combines game state representation, move generation, search algorithms, and a heuristic evaluation function. The goal of this system is to simulate possible future moves and select the one that leads to the most favorable outcome.

We used a Python-based chess library to handle the core game mechanics, including maintaining the board state, generating all legal moves, and detecting game-ending conditions such as checkmate or stalemate. This allowed us to focus on implementing the decision-making logic rather than building the game from scratch.

At a high level, the AI follows a repeated process on each turn. First, it generates all legal moves from the current board position. Then, for each possible move, it simulates the resulting board state and applies a search algorithm—either minimax or alpha-beta pruning—to explore future positions up to a fixed depth. Once the search reaches the depth limit or a terminal state, the evaluation function is used to assign a score to the board.

These scores are then propagated back up the search tree to determine which move leads to the best outcome, assuming both players make optimal decisions. The selected move is then executed on the board, and the process repeats for the next turn.

In addition to the core AI logic, we also developed both a command-line interface and a graphical user interface to allow interaction with the system. These interfaces support human vs. AI gameplay as well as AI vs. AI simulations, which were used to run experiments and collect performance data.

Overall, this approach provides a structured way to evaluate and compare different search strategies while maintaining a functional and interactive chess environment.

IV. Algorithms

IV.1. Minimax

The minimax algorithm is a foundational approach used for decision-making in two-player, turn-based games like chess. The main idea behind minimax is to simulate all possible moves up to a certain depth while assuming that both players are playing optimally.

In this setup:

- The AI acts as the maximizing player
- The opponent acts as the minimizing player

At each level of the search tree, the algorithm alternates between these roles:

- The maximizing player selects the move with the highest evaluation score
- The minimizing player selects the move with the lowest evaluation score

This process continues recursively until:

- A fixed depth limit is reached, or

- The game reaches a terminal state

At that point, a heuristic evaluation function is used to estimate the strength of the board position. These scores are then propagated back up the tree to determine the best move from the current position.

Key characteristics of minimax:

- Explores all possible moves up to a given depth
- Assumes optimal play from both players
- Guarantees the best possible move under those assumptions

However, minimax has a major limitation. The number of nodes it explores grows exponentially with search depth, which makes it computationally expensive and difficult to scale for more complex positions in chess.

IV.2 Alpha-Beta Pruning

Alpha-beta pruning is an optimization of the minimax algorithm that improves efficiency by eliminating branches of the search tree that do not need to be explored. It produces the same result as minimax but with significantly fewer computations.

The algorithm introduces two important values:

- Alpha (α): the best value the maximizing player can guarantee so far
- Beta (β): the best value the minimizing player can guarantee so far

As the search progresses, these values are updated. Pruning occurs when:

- $\alpha \geq \beta$, meaning further exploration of that branch cannot improve the result

When this condition is met, the algorithm stops exploring that branch entirely.

Key advantages of alpha-beta pruning:

- Reduces the number of nodes explored
- Maintains the same optimal decision as minimax
- Improves performance significantly, especially at deeper search levels

In our implementation, alpha-beta pruning follows the same recursive structure as minimax but includes these additional checks to skip unnecessary computations. This allows the AI to evaluate more promising moves while avoiding wasted effort on irrelevant branches.

Overall, alpha-beta pruning makes adversarial search much more practical for real-time gameplay, especially in a complex game like chess where the search space grows rapidly.

V. Evaluation Function

Since it is not practical to search all the way to the end of a chess game, the AI relies on a heuristic evaluation function to estimate how favorable a given board position is. This function assigns a numerical score to a position, where higher values indicate an advantage for White and lower values indicate an advantage for Black.

Our evaluation function considers several key factors:

- Material Balance
  Each piece is assigned a value.
  The total score is calculated by subtracting Black's material from White's material.
  This ensures that positions with stronger pieces are evaluated more favorably.

- Mobility
  Mobility is measured by counting the number of legal moves available to each player.
  Positions where a player has more options are generally stronger, so the evaluation rewards higher mobility.

- Center Control
  The function gives additional weight to pieces occupying central squares.
  Controlling the center is important in chess because it allows for better piece activity and flexibility.

- Terminal States
  Special cases are handled explicitly:
    - Checkmate results in a very large positive or negative score
    - Stalemate or insufficient material results in a neutral score

These components are combined into a single score that represents the overall strength of the board position. This score is then used by the search algorithms to compare different possible moves.

While this evaluation function is relatively simple compared to those used in advanced chess engines, it is effective enough to guide decision-making and allows us to clearly observe the impact of the search algorithms themselves.

VI. Experimental Setup

To evaluate the performance of minmax and alpha-beta pruning, we conducted a series of controlled experiments using AI vs. AI gameplay.
We tested both algorithms at search depths of 1, 2, and 3. For each configuration, the AI played full games against itself, and we recorded several performance metrics, including:

- Total runtime per game
- Average runtime per move

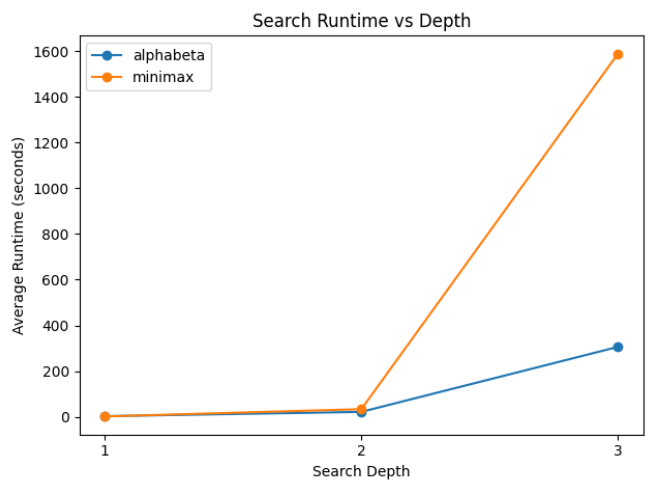
- Total number of nodes expanded
- Average number of nodes expanded per move

All experiments were implemented in Python using a chess library for moving generation and board management. The same evaluation function was used for both algorithms to ensure a fair comparison.

VII. Results and Analysis

The results clearly demonstrate the impact of search depth and the effectiveness of alpha-beta pruning.

Runtime Analysis



As shown in Figure X, the runtime of both algorithms increases significantly with search depth. However, minimax experiences a much steeper increase compared to alpha-beta pruning.

At depth 3, minimax takes approximately 1580 seconds, while alpha-beta pruning takes only about 310 seconds. This shows that alpha-beta pruning eliminates unnecessary branches, reducing the total number of positions that need to be evaluated.

Nodes Expanded Analysis

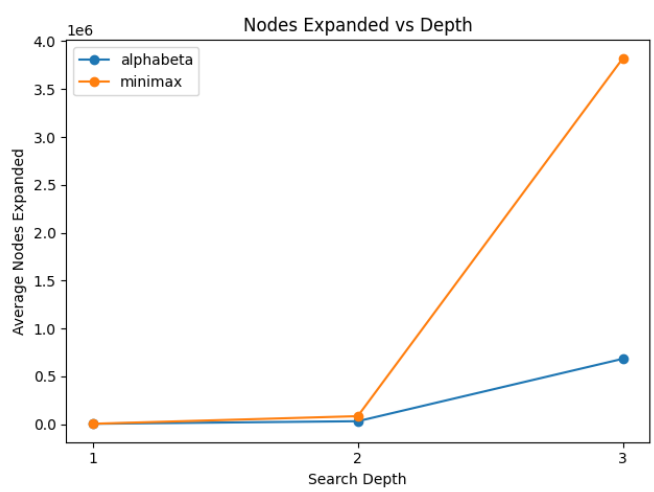


Figure Y shows the number of nodes expaned by each algorithm.

At depth 3:

- Minimax expands approximately 3.8 million nodes
- Alpha-beta expands about 0.7 million nodes

This confirms that alpha-beta pruning significantly reduces the size of the search tree.

Key Observatory

1. Exponential Growth

Both runtime and nodes increase rapidly as depth increases, highlighting the computational cost of deeper searches.

2. Efficiency of Alpha-Beta Pruning

Alpha-beta pruning consistently explores far fewer nodes than minimax, leading to substantial performance improvements.

3. Same Decision Quality

Despite the reduction in computation, alpha-beta pruning produces the same decisions as minimax, making it a more practical approach.

Conclusion from Results

These results demonstrate that alpha-beta pruning is essential for scaling adversarial search to deeper levels. Without pruning, minimax quickly becomes impractical due to the exponential growth of the search space.

IX. Conclusion

In this project, we implemented and compared two adversarial search algorithms, minimax and alpha-beta pruning, for a chess-playing AI. Both algorithms were able to make optimal decisions based on a heuristic evaluation function, but their performance differed significantly.

Our experimental results showed that while minimax explores all possible moves and quickly becomes computational cost. This highlights the importance of optimization techniques such as alpha-beta pruning in making adversarial search practical.

We also demonstrated that increasing search depth improves decision-making but comes at a high computational cost. This highlights that importance of optimization techniques such as alpha-beta pruning in making adversarial search practical.

Overall, this project illustrates the trade-off between search depth, computational efficiency, and decision quality in game-playing AI systems.

X. Future Work

There are several ways this project can be improved in the future. One possible extension is implementing better move ordering techniques to further improve the effectiveness of alpha-beta pruning. Another improvement would be incorporating iterative deepening, which allows the algorithm to search deeper within a time limit.

Additionally, move advanced evaluation functions could be developed, including machine learning-based approaches or more sophisticated positional heuristics. Finally, techniques such as transposition tables could be used to avoid re0evaluating previously seen positions, futher improving efficiency.